C47/R47 Solves, integration, plotting of RPN functions

ADV menu: RPN solves, integrals and plots

The RPN programmed function takes the traditional HP-42S approach: you write a short (RPN) program that evaluates a function or expression step by step on the stack, allowing loops, stores, recalls, advanced functions, sub-functions, etc. and lets you declare each variable used by the expression explicitly with MVAR.

EQN menu: algebraic solves, integrals and plots:

In contrast, the EQN menu lets you enter a formula in algebraic entry, much as it appears on paper; the calculator then parses that expression and exposes its variables automatically.

For the extra effort, the RPN route gives you full control over the computation. This appnote works through the RPN approach in detail. We show:

By hand, from the UI

- Run SOLVE , ∫f d and PLTf on one worked example, straight from the menus.

In an RPN program

- Automate and sequence the same solve, integrate and plot, then run them together.

The RPN programming techniques behind the demo

- Looping, indirect execution and output formatting used by the demo program.

The ‘Gauntlet of Integrals’: demo, benchmark and comparison

- Eleven hard integrals from Duncan Murray’s article, implemented as RPN programs,
- with the plots and integral accuracy compared against other calculators.
- An older graphing method (STATS/PLSTAT) is included in an annex for completeness.

Table of Contents

Manually solve and integrate on C47/R47

Step -1: What is MVAR?

MVAR stands for "menu variable." With an MVAR 'x' instruction in a program, you are telling the calculator that your RPN program will need a value for your variable used, say x. When you run SOLVE, PLTf, or ∫ dx, the C47/R47 (referred to as 47 below) then shows a variable menu with each declared MVAR variable. You press a number and a function key to store into that variable, or to recall what is currently held there with RCL.

Step 0: Start with an RPN equation

$f(x) = x^2 + 2x - 2 = 0$

Create an RPN program that evaluates the expression f(x) and then would solve, integrate or polt for f(x).

Step 1: RPN function/program

The RPN program referred to here is the formula itself. It declares variables using MVAR and uses RCL to place the required variable on the stack for evaluating the expression. This approach allows you to include any variables you want to appear in the solve menu, but it must include at least the variable to be solved for.

```
C47 Program file export: Export format version 3, C47 program version 1.
0000:  Prgm #2/2: 34 bytes / 13 steps

0001:  LBL 'FUNC'
0002:    MVAR 'x'
0003:    RCL 'x'
0004:    ENTER
0005:    x²
0006:    x⇄y
0007:    2
0008:    ×
0009:    +
0010:    2
0011:    -
0012:  END
```

Entering the program:

From run mode, press [XEQ ..] to open a new program space, then f [PRGM] to enter program mode. After entering the program, press f [PRGM] again to return to run mode.

To enter a typical RPN program for a quadratic expression's roots, type:

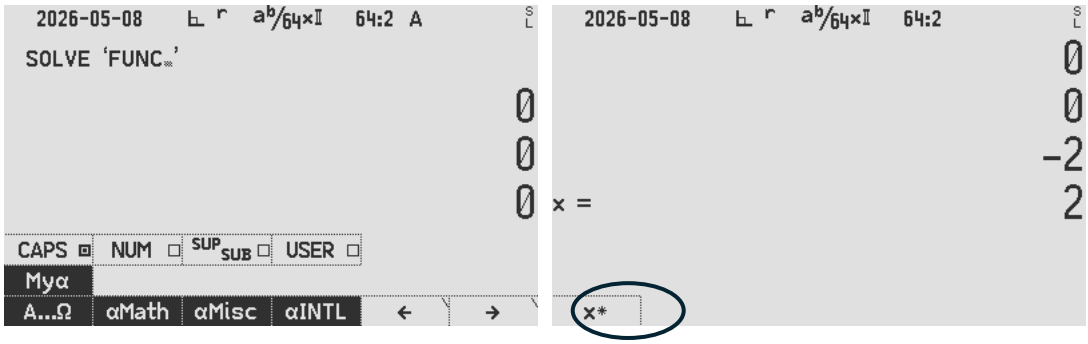
Type: XEQ . . PRGM LBL α FUNC MVAR α x ENTER x² x⇄y 2 × + 2 -

Keystrokes: [XEQ].. f[R/S] [F1] [XEQ] F U N C [F6] [F6] [F4] [XEQ] 1 ENTER f[x²] [x⇄y] 2×+2-

- The [MVAR 'x'] command instructs the 47 program to stop and display a menu when SOLVE, [∫f d] or PLTf is invoked.
- All declared MVAR variables will show in a menu, e.g. 'x' as declared in this example.
- After MVAR 'x' you must use RCL 'x' to use the value on the stack as usual.

Step 2: SOLVE: Manual solve for the roots

- Pick starting points for the solver, for example, set x from -2 to 2 to start the search for roots:



ADV SOLVE α FUNC ENTER 2 CHS [x] 2 [x] [x] STO 00

Root found, r1: x = 0.732 050 807 568 877 293 527 446 341 505 872 4 stored to R00

Each solution will return only one root. After finding a root, you must provide new starting values to guide the solver towards the next one, similar to the behaviour of the HP 11C, 15C, and 42S. Guide the solver to a different part of the function to search for a second root, e.g. pick more negative points to seek the other root:

10 CHS [x] 11 CHS [x] [x] STO 01

Root r2: x = -2.732 050 807 568 877 293 527 446 341 505 872 stored to R01

Step 3: ∫ f d: Manual integration to get the surface area

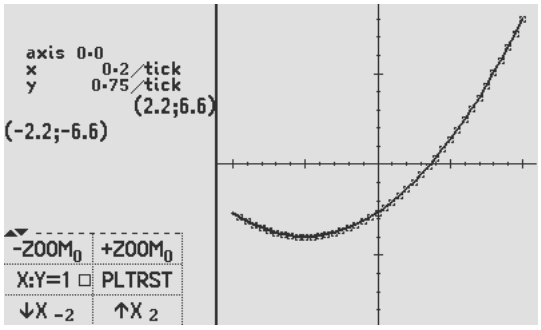
- Get the surface area between the curve and the x-axis, between the two roots:

∫f d α FUNC ENTER RCL 00 [x] RCL 01 [x] [x] $\int_y^x$ STO 02

Area: ∫x dx = 6.928 203 230 275 509 174 109 785 366 023 489 stored to R02

Step 4: PLTf: Manual graph/plot of MVAR RPN program/formula

- Plot a graph of the f(x) where f is the function and x are the MVAR variable.
- ADV PLTf α FUNC ENTER 2 CHS [x] 2 [x] [x]



Programmed roots, integrals and graphs

Programmed Roots

Create a program to automate root finding, store them (as above) to R00 & R01:

```
C47 Program file export: Export format version 3, C47 program version 1.
0000:  Prgm #1/1: 146 bytes / 35 steps

0001:  LBL 'INTDIF'
0002:    PGMSLV 'FUNC'
0003:    -2
0004:    2
0005:    SOLVE 'x'
0006:    STO 00      // store r1
0007:    -11
0008:    -10
0009:    SOLVE 'x'      // store r2
0010:    STO 01
```

Pro tip 1:

Create the second program within the programming space of the first. This keeps related functions together in one file when saving (WRITEP) or exporting (XPORTP).

Programmed integration

Create a program to find the area between the two roots found above:

```
0011:    PGMINT 'FUNC'
0012:    RCL 00      // lower limit being root1 in Y
0013:    RCL 01      // upper limit being root2 in X
0014:    ∫xy 'x'
0015:    STO 02      // store integral
0016:    'x²+2x-2:↵∫=↵r2=↵r1='
0017:    STO 04      // 4 lines of display text are stored in r04: Top↵X↵Y↵Z
0018:    CLSTK
0019:    RCL 00      // show root 1
0020:    RCL 01      // show root 2
0021:    RCL 02      // show integral
0022:    AVIEW 04    // show user text
0023:  RTN
```

Greyed out program already entered in the prior sections above:

```
0024:  LBL 'FUNC'
0025:    MVAR 'x'
0026:    RCL 'x'
0027:    ENTER
0028:    x²
0029:    x≷y
0030:    2
0031:    ×
0032:    +
0033:    2
0034:    -
0035:  RTN
```

Pro tip 2:

For quick access to commands, functions, and variables, type **α** followed by the name directly. For example after XEQ, SOLVE, ∫f d, PLTf, (marked in red). This is faster than navigating menus, though menu selection works equally well.

Programmed plotting of graphs

This section deals with the plotting of y = f(x) graphs, defined by your user RPN function `FUNC`, using the `PLTf` method:

The MVAR method used is the same as in the solve and integrate functions discussed above.

Create a small program in the same program space as the FUNC program above. Here we continue to enter it from step 36 in the listing above:

```
0036:  LBL 'GRAPH'
0037:    PGMPLT 'FUNC'
0038:    -2
0039:    2
0040:    PLTf 'x'
0041:    SNAP or OPENM 'PLFUNC' for interactive, or PAUSE 20 or PAUSE 99
```

Programmed PLTf plots the graph and stops with the graph visible at the end of the program. If there are more commands after PLTf, it will continue executing without stopping and the options of dealing with the graph are:

- **SNAP** to snap a picture (into a program by typing "`XEQ α S N A P ENTER`"). (SNAP needs at least a second delay before another SNAP, use PAUSE 10).
- **OPENM 'PLFUNC'** to open the left side menu (into a program by typing "`XEQ α O P E N M ENTER α P L F U N C`")
- **PAUSE nn**, a short pause to let the user see before continuing.

RPN programming techniques

INTDEMO — the demo program that runs the benchmark in the next section — is built from the RPN techniques below.

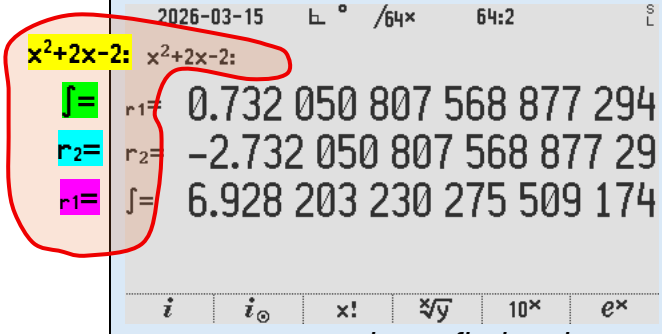
The 47 program referenced is used as a reference, and can convert to a p47 program file in both directions, using rejig. The program serves as a benchmark of the plotting and integration subsystems across easy and deliberately hard integrands (singularities, power towers, wild oscillation).

INTDEMO loops over eleven test functions, plotting each one and then numerically integrating it over its bounds, printing the integral, its error estimate and a relative-accuracy figure. This demo futher exercises the following 47 programming techniques:

- Alpha and numeric labels: (`LBL 'INTDEMO', LBL 00`)
- Counted loop: (`DSZ/GTO` (11 down to 1))
- Setting calculator mode: (`RAD`)
- Building label name at runtime: (`'BND_P' + RCL 00`),
- Executing indirectly: (`XEQ →R02`).
- Setting graph & integration limits on the stack from a subroutine.
- Plotting: `PGMPLT, PLT f 'x', PLTRST`
- Integration: `PGMINT, ∫xd`
- Output and formatting: `xy`
- Computing relative accuracy: $|x|$, $\log_{10}(x)$
- Function and stack use: $\sqrt{x}$, $1/x$, x^2, e^x, `tan, cos, sin, ln, Γ,` y^x
- User info `AVIEW`

Three of these techniques carry most of the program's structure and reward a closer look with the listing in hand (Annex A): the counted loop, indirect execution, and the formatting of printed output.

Display Temporary Information with output (AVIEW)



`INTDIF` *automated root find and output*

Pro tip 3:

Create custom text labels in step 0016 and label the stack elements in the sequence of Top↵X↵Y↵Z in step 0022 above. The mapping is shown in colours as follows:

x²+2x-2:↵ r1=↵r2=↵∫=

Looping with a counter (DSZ / GTO)

All eleven test cases are driven by one counted loop. Decrement Skip if Zero (DSZ) is used as one of the anchor loop structures. A counter is initialised once and decremented on each pass:

```
11
STO R00
LBL 00
  … loop body …
  DSZ R00
    GTO 00
  RTN
```

R00 is loaded with 11 and LBL 00 marks the top of the loop. At the bottom, DSZ R00 (“decrement, skip if zero”) subtracts one from R00 and skips the following GTO 00 only when the counter reaches zero; otherwise control jumps back to LBL 00. The body therefore runs eleven times, with R00 counting 11 → 1.

Indirect execution (run-time label names)

Rather than an eleven-way branch, INTDEMO builds the name of the routine it needs at run time and calls it indirectly. On each loop it concatenates a fixed string with the loop counter to form the target label:

```
'BND_P'
RCL R00
+
STO R02
  …
XEQ →R02
```

Pushing the alpha string 'BND_P', recalling the counter (say 7) and adding them with + yields the label name “BND_P7”, stored in R02. XEQ →R02 then executes the label whose name is held in R02. The arrow (→) is the indirect prefix, meaning “use the contents of the register, not the register itself.” The integrand routines are dispatched the same way: 'INT_P' + RCL R00 builds INT_P7 into R01, and PGMINT →R01 points the integrator at it.

One loop body thus services all eleven bound/integrand pairs (BND_P1···BND_P11, INT_P1...INT_P11) with no branch table.

Output formatting (🖨xy)

Results are written to the print/disk log with the 🖨xy command, which outputs a labelled line from the stack. The program interleaves a little arithmetic with string building to format each line:

```
RCL Z
÷
|x|
log10(x)
CHS
'Relative Accuracy:'
🖨xy
```

Here the error (in Z) is divided by the result, reduced to a magnitude with |x|, and passed through $\log_{10}$ then CHS — giving $-\log_{10}|\text{error} \div \text{result}|$, effectively the number of correct significant digits.

The alpha string 'Relative Accuracy:' supplies the line label, and 🖨xy prints value and label together. Units are tagged the same way elsewhere: a time value is divided by 10 and the literal ' s' is appended with + before printing, so the log reads, e.g., “8.5 s”.

Run a program (XEQ)

Get the result by running the program defined above: **type XEQ α INTDIF ENTER**

Automated integrals and performance testing

This section puts the 47's RPN integrator through a published 'Gauntlet' of eleven hard integrals. We first set out how accuracy is judged, then run the eleven problems with the 47's results, and finally compare the 47 against other calculators.

Evaluating the results

To evaluate the results, we use reference data, with more digits and more accuracy than the 34-digit calculator provides.

The accuracy figure is determined by comparing the error with the result, using $-\log_{10}(|\frac{error}{result}|)$. The accuracy calculation is done in two ways, namely:

1. using the claimed error and result from the integrator on the calculator, i.e. $-\log_{10}(|\frac{47\ error}{47\ result}|)$. This tests the full result that the calculator claims.

2. using the actual error and the actual reference result, i.e. $-\log_{10}\left(\left|\frac{47\ result - reference\ result}{reference\ result}\right|\right)$.
   This tests the accuracy of the actual error, ignoring the Y-value given which is at best an estimate.

The reference integral result values used for the 11 defined problems:

Problem	Reference	Source
P1	0.828 427 124 746 190 097 603 377 448 419 396 157 139 3	Duncan
P2	1.633 031 481 071 948 259 293 952 014 043 698 584 924 1	Duncan
P3	2 259	Duncan
P4	11.970 716 785 493 139 183 256 615 050 324 309 092 447 4	Duncan
P5	0.731 339 779 891 570 998 493 721 841 652 133 420 783	Duncan
P6	-0.946 083 070 367 183 014 941 353 313 823 179 657 812 3	Jaco, Wolframalpha
P7	2	Duncan
P8	1.772 453 850 905 516 027 298 167 483 341 145 182 797 5	Duncan
P9	0.918 938 533 204 672 741 780 329 736 405 617 639 861 3	Duncan
P10	2.996 339 811 976 535 379 182 861 556 235 501 887 668	Duncan
P11	0.390 992 162 151 530 454 531 709 880 804 974 485 310 3	Duncan

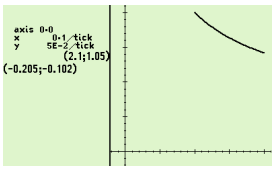
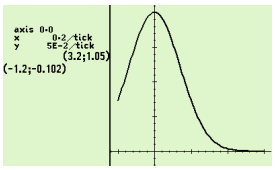
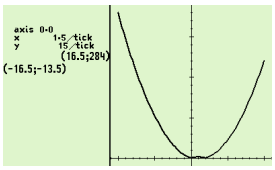
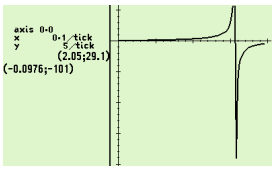
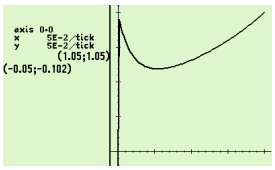
Test the 11 integrals P01 – P11 referred to in the SM Forum

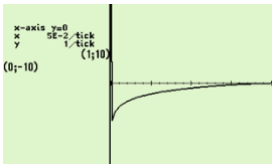
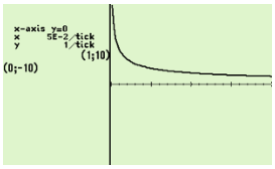
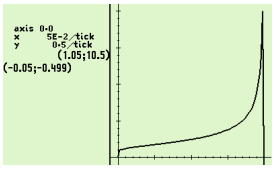
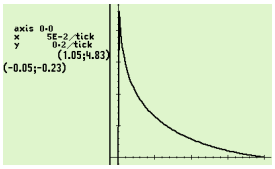
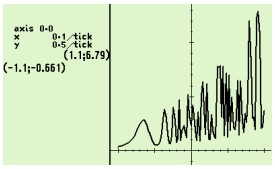
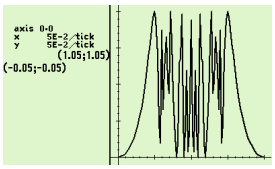
Below in Annex A is the code that runs these eleven integrals on the 47, with an analysis of the accuracy of the returned results.

Duncan Murray compiled a list of integrals, mostly from Valentin Albillo's most challenging definite integrals, as discussed on the SwissMicros Forum. The reason for this effort is to compare the different algorithms that different calculators use, for example, the 47 calculator employs the 'New DEI' method:

The double-exponential integrator in the 47 implements the improved scheme described by Graillat and Ménissier-Morain. This scheme benchmarks several different integrators and outperforms the older Michalski-Mosig approach it replaces. The translation runs in full double-precision arithmetic and can handle finite, semi-infinite, and doubly infinite limits. For semi-infinite integrals, a quick pre-pass identifies an optimal split point before the main quadrature begins, resulting in significant speed-up with minimal additional work.

Below the 11 integrals, complete with 47 result and 47 plot. The accuracy method and reference values are given under Evaluating the results above; the R47 values were confirmed 2025-06 and 2025-12.

Pnn	Text	Equation	
P01	∫ from 1 to 2 of 1/sqrt(x) dx Result: 0.828427124746190097603377448419396	$\int_1^2 \frac{1}{\sqrt{x}}\,dx$	
P02	∫ from -1 to 3 of e^(-x^2) dx Result: 1.633031481071948259293952014043699	$\int_{-1}^{3} e^{-x^2}\,dx$	
P03	∫ from -15 to 15 of |x^2 - 3x| dx Result: 2258.99338820680346729497256898164 51	$\int_{-15}^{15} \lvert x^2 - 3x \rvert\,dx$	
P04	∫ from 0 to 1.57079 of tan(x) dx Result: 11.970716785493139183256615050330 89	$\int_0^{1.57079} \tan x\,dx$	
P05	∫ from 0 to 1 of x↑↑4 dx (i.e. x^(x^(x^x))) Result: 0.731339779891570998493721841652133 4	$\int_0^1 x^{x^{x^x}}\,dx$	

Pnn	Text	Equation	
P06	∫ from 0 to 1 of cos(x) * ln(x) dx Result: -0.946083070367183014941353313823179 6	$\int_0^1 cos\,x\,ln\,x\;dx$	
P07	∫ from 0 to 1 of 1/sqrt(x) dx Result: 2	$\int_0^1 \frac{1}{\sqrt{x}}\,dx$	
P08	∫ from 0 to 1 of 1/sqrt(-ln(x)) dx Result: 1.772453850905516013219002902374754	$\int_0^1 \frac{1}{\sqrt{-ln\,x}}\,dx$	
P09	∫ from 0 to 1 of ln(Gamma(x)) dx Result: 0.918938533204672741780329736405617 6	$\int_0^1 ln\,\Gamma(x)\;dx$	
P10	∫ from -1 to 1 of e^(x + sin(e^(e^(e^(x + 1/3))))) dx Result: 2.969857419600580236632783813721218	$\int_{-1}^1 e^{x+sin\,e^{e^{e^{x+\frac{1}{3}}}}}\,dx$	
P11	∫ from 0 to 1 of sin^2(tan(tan(pi*x))) dx Result: 0.400962473290370067589052596165374 9	$\int_0^1 sin^2(tan\,tan\,\pi x)\,dx$	

47 Numerical Results: 47 & Simulator 00.109.03.02b, dd. 1 May 2025, cdaa2bf

Agrees with 2025-12-12 version `c9249ddd3` tests by Ralf.

Pnn	47-model	47 result	47 claimed accuracy	R47 Time	Self Rel. Error	Ref. Rel. Error
P1	C47 SIM R47 Hardware	0.828427124746190097603377448419396 1	4.778068138718563818069779722687834E-35	2.7 s 8.5 s	34.2	34.2
P2	C47 SIM R47 Hardware	1.633031481071948259293952014043699	6.372351127713208137933736854265888E-36	2.7 s 10.4 s	35.4	33.6
P3	C47 SIM R47 Hardware	2258.993388206803467294972568816451	0.000014044495490583500891720554825051 5	10.5 s 37.1 s	8.2	5.5
P4	C47 SIM R47 Hardware	11.970716785493139183256615050330 89	1.840560392346996688334359177881751E-31	8.9 s 49.5 s	31.8	30.3
P5	C47 SIM R47 Hardware	0.731339779891570998493721841652133 4	1.083200342414643890366907758364657E-35	2.4 s 31.1 s	34.8	34.5
P6	C47 SIM R47 Hardware	-0.946083070367183014941353313823179 6	2.873360263116166866379212992769967E-35	2.2 s 20.9 s	34.5	34.2
P7	C47 SIM R47 Hardware	2.	1.824712499999999999999999999999997E-33	1.2 s 4.8 s	33.0	∞
P8	C47 SIM R47 Hardware	1.772453850905516013219002902374754	3.098338881919348930600803772417547E-19	8.4 s 62.0 s	18.8	17.1
P9	C47 SIM R47 Hardware	0.918938533204672741780329736405617 6	6.618709826857734469045946487296794E-35	2.6 s 44.3 s	34.1	34.4
P10	C47 SIM R47 Hardware	2.969857419600580236632783812721218	0.018433880549434384279330569653158 91	8.8 s 100.9 s	2.2	2.1

Pnn	47-model	47 result	47 claimed accuracy	R47 Time	Self Rel. Error	Ref. Rel. Error
P11	C47 SIM R47 Hardware	0.400962473290370067580952596165374 9	0.049478818095536602769177731254971 09	6.9 s 75.5 s	0.9	1.6

Red indicates significant difference in the self-ascertained 'error'.

Self Relative Error $= -\log_{10}(|\frac{47\ error}{47\ result}|)$ ***Reference Relative Error*** $= -\log_{10}\left(\left|\frac{47\ result - reference\ result}{reference\ result}\right|\right)$,

Comparative results: Source Duncan Murray, SwissMicros Forum

calculator	P01	P02
Ref	0.828427124746190097603377448419396157139	1.633031481071948259293952014043698584924
WP43	0.828427124746190097603377448419396	1.633031481071948259293952014043699
Casio fx-115W	0.82843	1.633
Sharp EL-W506T	0.828427124	1.633031482
TI-83 Plus	0.8284271247	1.633031481
Casio fx-9910CW		
TI-NSpire CX II KhiCAS 2.0		
HP Prime 2		
TI-89 Ti		
Numworks 24.10.0	0.82842712474619	1.6330314810719
R47	0.828427124746190097603377448419396 1	1.633031481071948259293952014043699
TI-NSpire CX II		
DM-15L FIX 4	0.8284	1.6330
db48x 0.9.14 S12	0.828442760473	1.63303136781
db48x 0.9.14 S24a	0.828426601063264722661203	1.633031367869810981404 61
db48x 0.9.14 S24b	0.828427124746188196214920	1.633031481071948265140 48
db48x 0.9.14 S48	0.828427124746190097982799267593872004769254	1.633031481071948265140737104583222732691 53
db48x 0.9.14 Sstd	0.828427124746190097982713	1.633031481071948265140 48
HP-34C SCI 9	0.8284271248	
DM-15L SCI 8	0.8284271248	1.633031481
Casio fx-991EX	0.82842712474618	1.63303148110593

calculator	P03	P04
Ref	2259	11.970716785493139183256615050324309092447 4
WP43	2258.9933882068034672949725688164 53	11.970716785493139183256615050330 89
Casio fx-115W	2250	ERR
Sharp EL-W506T	2259	419.4464408
TI-83 Plus	2250	11.97071678
Casio fx-9910CW	2259	11.9707167854931
TI-NSpire CX II KhiCAS 2.0		11.970716784 1
HP Prime 2		11.970716784 1
TI-89 Ti	2259	11.970716784959
Numworks 24.10.0	2259	11.970716785491
R47	2258.993388206803467294972568816451	11.970716785493139183256615050330 89
TI-NSpire CX II	2250	
DM-15L FIX 4	2258.9998	11.9707
db48x 0.9.14 S12	2259.11276851	11.9706034618
db48x 0.9.14 S24a	2259.00218853262525219728	11.9707171781668250584788
db48x 0.9.14 S24b	2258.99999999892401571912	11.970716785493144792191 7
db48x 0.9.14 S48	ERR	11.970716785493139182676745245506165896476 5
db48x 0.9.14 Sstd	ERR	ERR
DM-15L SCI 8	2259.000006	ERR
Casio fx-991EX	2250	11.9707167841225

calculator	P05	P06

Ref	0.731339779891570998493721841652133420783	-0.946083070367183014941353313823179657812
WP43	0.731339779891570998493721841652133	-0.94608307036718301494135331382317
Casio fx-115W	ERR	ERR
Sharp EL-W506T	ERR	ERR
TI-83 Plus	0.7313394496	-0.9460814174
Casio fx-9910CW		
TI-NSpire CX II KhiCAS 2.0		
HP Prime 2		
TI-89 Ti		
Numworks 24.10.0	0.73133977989157	-0.94608307036718
R47	0.731339779891570998493721841652133	-0.946083070367183014941353313823179
TI-NSpire CX II		
DM-15L FIX 4	0.7313	-1.0000
db48x 0.9.14 S12	0.940522403160	-0.945912828691
db48x 0.9.14 S24a	0.940318095892148559294545	-0.946072479866791647174107
db48x 0.9.14 S24b	0.940318086681907169857535	-0.946083070367025410749963
db48x 0.9.14 S48	0.940318086681907169857608447182094166044772	ERR
db48x 0.9.14 Sstd	0.940318086681907169857535	ERR
HP-34C SCI 9		
DM-15L SCI 8	0.7313397799	-0.9460830688
Casio fx-991EX	ERR	ERR

calculator	P07	P08
Ref	2	1.772453850905516027298167483341145182797
WP43	2	1.772453850905516013219...
Casio fx-115W	ERR	ERR
Sharp EL-W506T	ERR	ERR
TI-83 Plus	1.999994424	1.772446281
Casio fx-9910CW		
TI-NSpire CX II KhiCAS 2.0		
HP Prime 2		
TI-89 Ti		
Numworks 24.10.0	2	1.7724538406522
R47	2	1.772453850905516013219002902374754
TI-NSpire CX II		
DM-15L FIX 4	1.9999	1.772324417
db48x 0.9.14 S12	1.99896964912	1.77142350570
db48x 0.9.14 S24a	1.99987120626587583845160	1.77232505722451111834911
db48x 0.9.14 S24b	ERR	ERR
db48x 0.9.14 S48	ERR	ERR
db48x 0.9.14 Sstd	ERR	ERR
HP-34C SCI 9		
DM-15L SCI 8	ERR	ERR
Casio fx-991EX	ERR	ERR

calculator	P09	P10
Ref	0.918938533204672741780329736405617639861	2.996339811976535379182861556235501887668
WP43	0.91893853320467274178032973640561763	2.975...
Casio fx-115W	ERR	ERR
Sharp EL-W506T	ERR	ERR
TI-83 Plus	ERR	ERR
Casio fx-9910CW		

TI-NSpire CX II KhiCAS 2.0		
HP Prime 2		
TI-89 Ti		
Numworks 24.10.0	ERR	ERR
R47	0.9189385332046727417803297364056176	2.96985741960058023663278381272121 8
TI-NSpire CX II		
DM-15L FIX 4	0.9189	3.002381936
db48x 0.9.14 S12	0.918927942647	ERR
db48x 0.9.14 S24a	0.91893853062244762912960 3	ERR
db48x 0.9.14 S24b	ERR	ERR
db48x 0.9.14 S48	ERR	ERR
db48x 0.9.14 Sstd	ERR	ERR
DM-15L SCI 8	0.9189385317	ERR
Casio fx-991EX	ERR	ERR

calculator	P11
Ref	0.3909921621515304545317098808049744853103
WP43	0.398...
Casio fx-115W	0
Sharp EL-W506T	ERR
TI-83 Plus	ERR
Casio fx-9910CW	
TI-NSpire CX II KhiCAS 2.0	
HP Prime 2	
TI-89 Ti	
Numworks 24.10.0	0.35816045512655
R47	0.4009624732903700675809525961653749
db48x 0.9.14 (prec 42)	
TI-NSpire CX II	
DM-15L FIX 1	0.4169042624
DM-15L FIX 2	0.3916722662
DM-15L FIX 3	0.3955440737
DM-15L FIX 4	0.3911292357
DM-15L SCI 8	ERR
db48x 0.9.14 S12	ERR
db48x 0.9.14 S24a	ERR
db48x 0.9.14 S24b	ERR
db48x 0.9.14 S48	ERR
db48x 0.9.14 Sstd	ERR
Casio fx-991EX	ERR

Conclusion

Across the eleven benchmark integrals P01–P11, the C47/R47's RPN-programmed ∫f integrator returned full or near-full precision on most problems: P01, P02, P04, P05, P06, P07 and P09 each agreed with the reference values to roughly 30–34 significant digits, with P07 exact. This places the 47 alongside the high-precision models, and ahead of every fixed-precision pocket calculator in the comparison, many of which returned ERR on the harder integrands.

A few problems exposed the limits of the built-in integrator. P03 and P08 dropped to about 5 and 17 correct digits respectively, while P10 (~2 digits) and P11 (~1.6 digits) were the weakest albeit for pathological cases. For P11 the 47 returned 0.40096 against a reference of 0.39099. These cases share sharply peaked or near-singular integrands, where a fixed sample set under-resolves the peak; tightening the requested accuracy or splitting the interval around the peak is the practical remedy.

In short, the RPN-programmed route delivers reference-grade accuracy on well-behaved integrals at the cost of a few seconds to roughly 100 s of run time on hardware, and degrades predictably on pathological integrands rather than failing outright.

Because of the same MVAR-based user function that drives SOLVE, ∫f and PLTf, a single program can serve for root-finding, integration and plotting alike.

Annex A: R47 rejig Program Listing

```
LBL 'INTDEMO'
  RAD
  SF 'SSIZE8'


  11              # Prep: loop from 11 to 1
  STO R00
  LBL 00

# == LOOP START ==
    'BND_P'       # Prep: indirect bounds
    RCL R00
    +
    STO R02


    'INT_P'       # Prep: indirect integr.
    RCL R00
    +
    STO R01

# == SET AND DRAW GRAPH ==
    XEQ →R02      # Set graph bounds
    PLTRST        # Reset graph settings
    PGMPLT →01    # Set graph RPN eq.
    PLTf 'x'      # Do RPN graph plot(x)
    SNAP
    PAUSE 10
    # USE BYPASS TO SKIP INTEGRATION
    # GTO 'BYPASS'


# == SET AND INTEGRATE ==
    XEQ →R02      # Set integral bounds
    PGMINT →R01   # Set integration eq.
    ∫^x_y 'x'     # Do integration
    LASTT?        # Push integration time

    RCL Y         # Get result
    RCL R01       # Get function number
    🖶xy          # Output #No. & Result

    R↓ R↓         # Roll down time & acc.
    10 ÷ ' s' +   # Get seconds
    RCL Z         # Get declared accuracy
    🖶xy          # Output accuracy & time

    RCL Z         # Get declared accuracy
    ÷ |x| log10(x) CHS
    'Relative Accuracy:'
    🖶xy          # Output accuracy


  LBL 'BYPASS'
# == END LOOP ==
    DSZ R00
      GTO 00
    RTN


  LBL 'BND_P1'
    1
    2
    RTN

  LBL 'INT_P1'
    # f(x) = 1/√x,
    #    bounds [1, 2]
    MVAR 'x'
    RCL 'x'
    √x 1/x
    RTN
```

```
LBL 'BND_P2'
  -1
  3
  RTN

LBL 'INT_P2'
  # f(x) = e^(-x²),
  #    bounds [-1, 3]
  MVAR 'x'
  RCL 'x'
  x² CHS e^x
  RTN


LBL 'BND_P3'
  -15
  15
  RTN

LBL 'INT_P3'
  # f(x) = |x² - 3x|,
  #    bounds [-15, 15]
  MVAR 'x'
  RCL 'x' ENTER
  x² x≷y 3 × - |x|
  RTN


LBL 'BND_P4'
  0
  1.57079
  RTN

LBL 'INT_P4'
  # f(x) = tan(x),
  #    bounds [0, 1.57079]
  MVAR 'x'
  RCL 'x' tan(x)
  RTN


LBL 'BND_P5'
  0
  1
  RTN

LBL 'INT_P5'
  # f(x) = x^(x^(x^x)),
  #    bounds [0, 1]
  MVAR 'x'
  RCL 'x'
  ENTER  ENTER  ENTER
  y^x  y^x  y^x
  RTN


LBL 'BND_P6'
  0
  1
  RTN

LBL 'INT_P6'
  # f(x) = cos(x)·ln(x),
  #    bounds [0, 1]
  MVAR 'x'
  RCL 'x'
  cos(x)
  RCL 'x'
  ln(x)
  ×
  RTN
```

```
LBL 'BND_P7'
  0
  1
  RTN

LBL 'INT_P7'
  # f(x) = 1/√x,
  #    bounds [0, 1]
  MVAR 'x'
  RCL 'x' √x 1/x
  RTN


LBL 'BND_P8'
  0
  1
  RTN

LBL 'INT_P8'
  # f(x) = 1/√(-ln(x)),
  #    bounds [0, 1]
  MVAR 'x'
  RCL 'x' ln(x) CHS √x 1/x
  RTN


LBL 'BND_P9'
  0
  1
  RTN

LBL 'INT_P9'
  # f(x) = ln(Γ(x))
  #    bounds [0, 1]
  MVAR 'x'
  RCL 'x' Γ(x) ln(x)
  RTN


LBL 'BND_P10'
  -1
  1
  RTN

LBL 'INT_P10'
  # f(x)=e^(x+sin(e^e^e^(x+1/3)))
  #    bounds [-1, 1]
  MVAR 'x'
  RCL 'x' 1 3 / +
  e^x e^x e^x SIN
  RCL 'x' +
  e^x
  RTN

LBL 'BND_P11'
  0
  1
  RTN

LBL 'INT_P11'
  # f(x) = sin²(tan(tan(πx)))
  #    bounds[0,1]
  MVAR 'x'
  RCL 'x' π × tan(x) tan(x) sin(x) x²
  RTN


.END.
```

Process: Above programs were written in the 47 editor, saved, converted by rejig to plain text. Comments were added in free form text, re-converted back to 47.

Annex B: Alternative graphing — PLSTAT

The C47/R47 calculator has a dedicated plotting function for user RPN programs in addition to the EQN menu designed to plot algebraic formulas.

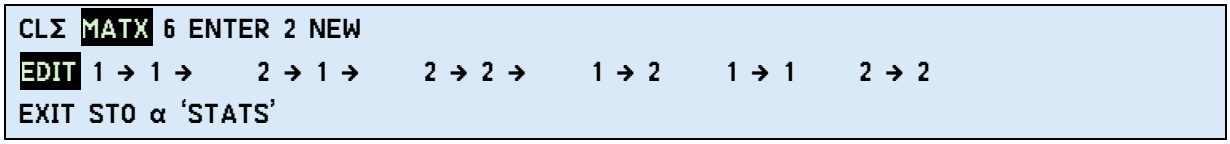
Plot (x, y) graphs using the STAT and PLOT menus

This method allows you to store a matrix of points via the Σ+ entry system; the concept is to store coordinates and connect the dots with lines. This method is employed in older demo programs such as BinetV3.p47.

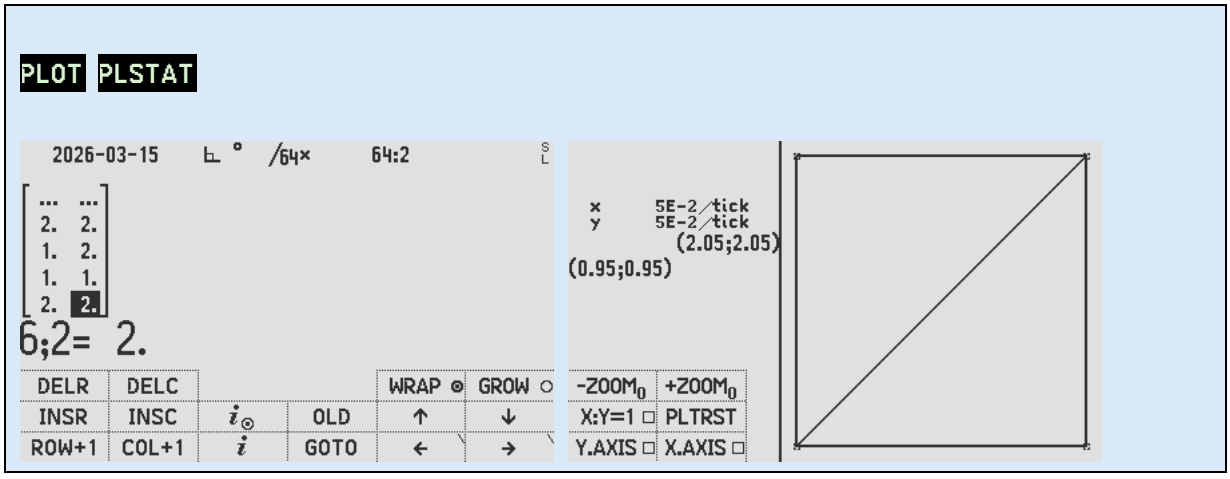
To plot a graph, use the 47 statistical functions. This involves entering (x,y) pairs into the STATS matrix. You can fill this matrix using the Σ+ or M.EDIT functions. Once the matrix is filled with coordinates, you can plot the graph using PLSTAT, which will draw lines between the points.

First, the principle using PLSTAT:

To manually plot from the STATS matrix, create a 6x2 matrix and populate it with a graph resembling a square and diagonal. Follow the example below carefully, entering the keypresses as you go.



Edit a new matrix, exit the editor, store to STATS*, and plot using* PLSTAT



Plot using PLSTAT

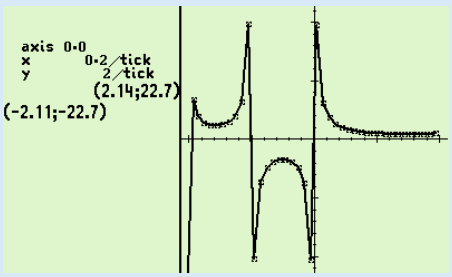
Graph an RPN function using PLSTAT:

Load the provided BinetV3.p47 program (in the PROGRAMS folder of the distro zip file) to see how this can be done in the same way as above, by calculating and populating graph coordinates into the STATS matrix and plotting it.

Annex C: Controlling graph parameters

You can pre-set the graph parameters before PLTf. See example below:

```
LBL 'GRAF-G'
  PGMPLT 'FUNC'
  -2
   2
   PLTRST       # Resets to default settings (find op P.FN1)
   20.
   STO '↑Y'       # Y upper value
   CHS
   STO '↓Y'       # Y lower value
   0
   ZOOM         # Set zoom factor, example shows 2
   CF PLnvec      # Set vector plotting mode (nav) with N = 0° clockwise
   CF PLvec       # Set vector plotting mode (math)
   CF PLxy:1      # Set symmetrical scales on x and y to be 1:1
   SF PLx.ax      # Set y-axis to be on screen
   SF PLy.ax      # Set x-axis to be on screen
   CF PLplus      # Set point marker to a plus
   CF CFcross       # Set point marker to cross
   SF PLbox       # Set point marker to box
   CF PLcurve       # (disabled)
   CF PLcxp       # Plot complex plot
   CF PLimp       # Plot imaginary part
  PLTf 'x'
  SNAP
RTN
LBL 'GAMMA'
  MVAR 'x'
  RCL 'x'
  Γ(x)
RTN
```



Author Jaco Mostert; proofread by Duncan Murray